

BLISK: Boolean circuit Logic Integrated into the Single Key

Oleksandr Kurbatov¹, Yevhen Hrubiiian¹, Illia Melnyk¹, Lasha Antadze²

¹ Distributed Lab

² Rarilabs

January 25, 2026

Abstract

This paper introduces **BLISK**, a framework that compiles a monotone Boolean authorization policy into a single signature verification key, enabling only the authorized signer subset to produce the standard constant-size aggregated signatures. BLISK combines (1) n -of- n multisignatures to realize conjunctions, (2) key agreement protocols to realize disjunctions, and (3) verifiable group operations (for instance, based on the 0-ART framework¹). BLISK avoids distributed key generation (allowing users to reuse their long-term keys), supports publicly verifiable policy compilation, and enables non-interactive key rotation.

1 Introduction

The secure control of cryptographic keys under shared authorization policies is a foundational challenge in distributed systems, especially in digital asset custody. In many real-world settings, control over an operation or process must be shared among multiple parties according to a policy that is neither purely centralized nor a simple threshold. A typical example of such a policy is “two out of three executives must approve, provided the compliance officer participates,” instead of pure multisignature.

Today, most deployed systems use threshold signature schemes, which allow any k of n parties to jointly produce a signature verifiable under a single public key. While threshold signatures are elegant and efficient (and increase liveness guarantees if some signers are denied), they suffer from several fundamental limitations.

First, they support only a single, fixed access structure of the form k -of- n . Let’s conduct a thought experiment and try to implement the following policy using only the k -of- n signature construction:

To consider the transaction valid, it:

1. Must be signed by Alice (A) or Bob (B) (1-of-2), and
2. Must be signed by Carol (C) or Dave (D) (one more 1-of-2)

The experiment yields the following observation: the authorization policy $(A \vee B) \wedge (C \vee D)$ cannot be realized by any standard or weighted threshold signature scheme.

Corollary 1.0.1. *Threshold schemes express only cardinality-based access structures and cannot enforce structured constraints across disjoint signer groups. Thus, threshold constructions are insufficient to express asymmetric or complex hierarchical policies.*

Second, threshold schemes require a distributed key generation (DKG) procedure—an interactive protocol that must be executed by all participants and repeated whenever the participant set or threshold changes (to be precise, even if only one party from the quorum lost the key but is still required by the policy, we need to involve the entire quorum and repeat DKG). Additionally, DKG produces fresh key material, preventing users from reusing existing wallets or long-term public keys. These limitations significantly complicate deployment in custody, governance, and other environments.

¹Basically, BLISK is the idea that follows the idea of 0-ART, extending the verifiable key agreement protocol for custom boolean conditions

An alternative approach is to enforce complex authorization logic at the verification layer, for example, by embedding scripts or multiple verification keys on-chain [Gno18; Bit09]. But obviously, this approach leads to another set of disadvantages and limitations:

- **Increased verification costs and witness sizes.** Modern multisignature and threshold signature schemes enable efficient (constant-size) aggregation of public keys and signatures. With on-chain policy verification, we revert to verifying a batch of single signatures.
- **Policy structure leakage.** The script

`<AlicePK> OP_CHECKSIGVERIFY 1 <BobPK> <CarolPK> 2 OP_CHECKMULTISIG`

says clearly what’s happening and who is responsible for operating the specific UTXO. The same problem applies not only to Bitcoin; if the smart account implements policy management, that policy is visible to anyone.

Our approach. In this work, we take a different perspective. Rather than modifying the signing or verification algorithms to support richer policies, we asked ourselves, *“Can an expressive authorization policy be compiled into a single public key, such that any authorized subset of parties can produce a standard aggregated signature for it?”*.

That’s how we came to **BLISK**, a cryptographic construction that compiles Boolean authorization policies over existing public keys into a single (public) signature verification key. The resulting key can be combined with a multisignature scheme (Schnorr-based or BLS), without modifying the signature verification logic (and introducing new attack vectors).

At a high level, our construction relies on two observations:

1. *Multisignature n -of- n schemes* naturally implement logical AND ($\wedge$): all participating signers must contribute for a signature to be valid.
2. *Key agreement protocols* allow a group of participants to derive a shared secret such that any one member can compute it, effectively implementing logical OR ($\vee$).

By combining these two primitives, we can construct a policy circuit in which each internal node represents either an AND or an OR gate, and each input wire corresponds to an existing user’s public key. Each node of the circuit is deterministically mapped to a public key, and the root key serves as the sole verification key for signatures generated under the specified policy. After the policy circuit is created and resolved, the signing process follows a standard multisignature protocol among the parties assigned to the policy.

Contributions. We summarize our contributions as follows:

1. We introduce policy circuits, a model for compiling Boolean policies into cryptographic key material.
2. We show how policy circuits can be combined with multisignature protocols to receive expressive authorization policies with efficient key aggregation.
3. We describe a mechanism for verifiable key rotation and membership changes that require no global circuit redeployment and are non-interactive.
4. We provide a reference implementation of the compiler for policy circuits based on BLISK.
5. We provide a security proof sketch of $\Sigma_{\text{BLISK}}^{\text{MuSig2}+0\text{-ART}}$ instance of **BLISK** protocol, which extends the standard signature (MuSig2) and key agreement scheme (0-ART) with policy circuit.

This paper is organized as follows. Section 2 provides an overview of related protocols and approaches. Section 3 consists of preliminaries and formal definitions of underlying primitives. We provide details of BLISK authorization circuits and BLISK multisignature protocol in Section 4. Section 5 includes discussions around the BLISK. Section 6 provides details of the reference implementation of the BLISK compiler. We provide an exact example of a BLISK signature instance, configured with MuSig2, 0-ART, and a specific policy, in Appendix A to demonstrate the appropriate security model.

2 Related work

Multisignatures. Efficient n -of- n multisignature schemes, such as Schnorr-based constructions (e.g., MuSig2 [NRS20]) and pairing-based schemes (e.g., BLS [BLS01]), allow a fixed set of parties to jointly produce a compact signature verifiable under a single aggregated public key. These schemes are typically analyzed under strong notions of unforgeability and rely on well-studied hardness assumptions, while offering minimal verification overhead and strong compatibility with existing systems. Conceptually, n -of- n multisignatures realize a logical conjunction: a valid signature can be produced if and only if all designated participants contribute.

Our work does not modify or weaken these schemes. Instead, we reuse n -of- n multisignatures as a primitive, preserving their original security guarantees, and extend their applicability by embedding them into a higher-level construction that supports general Boolean authorization policies. In particular, we combine conjunctions realized by multisignatures with disjunctions realized through key agreement, enabling expressive access structures while retaining single-key verification and standard unforgeability guarantees.

Threshold signatures. Threshold signature schemes allow a set of n parties to jointly produce a signature such that any subset of size at least k can sign, while smaller subsets cannot. A defining property of threshold signatures is that all valid signatures are verified under a single public key, making them attractive for blockchains and other cost-sensitive verification environments.

There are many approaches to efficient threshold signature schemes: ECDSA-based [GGN16; GG19; Hai+18], Schnorr-based [Gen+99; Gen+07], BLS [Bol02; DR23], etc. However, a fundamental limitation shared by all classical threshold signature schemes is that they support only a single policy of the form k -of- n . This restriction precludes mandating participation by any specific party, although partial adaptations are possible.

One such adaptation is provided by weighted threshold schemes [24], which allow the secret to be distributed among parties with different weights. As a result, a given participant may account for a majority of the total weight, thereby making their participation necessary to obtain a valid signature. However, this construction only allows assigning numerous shares to a single participant and doesn't resolve the significant limitation of threshold signatures.

Our work differs from threshold signatures in two key aspects: it does not require DKG, and supports general monotone boolean authorization policies while preserving single-key verification.

Linear Secret Sharing Schemes. Linear secret sharing schemes (LSSS) [Bei25] are a relatively old primitive in which the secret is an element of a finite field, and the shares are a linear combination of the secret and random elements from the field. These schemes often realize access structures represented by a monotone formula, such as CNF or DNF. Of particular interest to our work are monotone formulas, which are highly effective for defining diverse policies (basically, any policy can be described by a logical operation).

However, LSSS-based approaches are poorly suited to efficient signature aggregation. First, they typically require distributing multiple shares per participant, increasing storage and communication costs. Second, signing and verification costs scale with the size of the access structure, rather than remaining constant. Finally, dynamic updates to the access structure generally require redistributing shares or rerunning key generation procedures.

In contrast, we aim to support expressive monotone policies while producing constant-size signatures verifiable under a single public key, without redistributing secret shares.

Multi-Party Computation. MPC protocols [Lin20] are a powerful primitive that enables distributed computation among different but related parties. Many different techniques and schemes have been developed for constructing MPC protocols [EKR18]. Formally, MPC allows the construction of protocols for computing any shared function.

However, MPC fundamentally differs from policy-based approaches, which restrict participation to specific user subsets. Moreover, the protocols are often complex and require substantial interaction, coordination, and computational resources.

Ring Signatures and OR Constructions In [kAN20], Monero proposes a multisignature scheme in which combinations of various coalitions form a structure that closely resembles our approach. The construction begins with trivial threshold access structures, such as 1-of- n and $(n-1)$ -of- n . The latter is implemented using Diffie-Hellman shared secrets: each user computes $(n-1)$ shared private keys with every other participant, yielding a total of $n \cdot (n-1)$ keys.

By extending this design with aggregation techniques, they derive a threshold group aggregation algorithm for multisignatures. Furthermore, they briefly discuss the possibility of embedding and extending this family of algorithms to support policy specifications. However, this direction is not explored further in their work. We try to systematize this idea by combining disjunction realized through group key agreement with conjunction realized through multisignature aggregation.

Tree-Based Group Key Agreement Tree-based continuous group key agreement (CGKA) protocols, such as ART (asynchronous ratchet trees) [Alw+20] and TreeKEM [Bar+19], are designed to maintain a shared group secret under dynamic membership. A key property of these protocols is that any group member can derive the same group key, while non-members cannot.

We leverage this property in a novel way: instead of using CGKA solely for secure messaging, we use it to implement logical OR gates in authorization policies. To the best of our knowledge, this is the first work to use CGKA as a building block for compiling authorization logic into signing keys.

Another work, the so-called 0-ART [Hru+25], is dedicated to verifiable operations (e.g., update, delete) in ARTs. By adopting this approach, we can make all updates to authorization circuits verifiable, which is particularly important for trustless signer environments.

3 Preliminaries

Denote $\mathcal{Q}$ as a quorum of signers with $|\mathcal{Q}| = n$. $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ a hash function modeled as a random oracle. We can label the hash function instance with the tag $\mathcal{H}_{\text{tag}}$ to facilitate clear domain separation.

3.1 Underlying Primitives

Definition 3.1. We define an efficiently aggregatable multi-party signature scheme as the set of algorithms

$$\Sigma = \{\text{Setup}, \text{KeyGen}, \text{KeyAgg}, \text{SigGen}, \text{SigAgg}, \text{SigVerify}\},$$

where:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}_\Sigma$ initializes public parameters
- $\text{KeyGen}(\text{pp}_\Sigma) \rightarrow (\text{sk}, \text{pk})$ generates a signing key pair
- $\text{KeyAgg}(L) \rightarrow \text{pk}_\wedge$ aggregates public keys $L = \{\text{pk}_i : i \in \mathcal{Q}\}$ into a single public key $\text{pk}_\wedge$
- $\text{SigGen}(\text{sk}_i, m) \rightarrow \sigma_i$ generates a signature share of the user $i \in \mathcal{Q}$
- $\text{SigAgg}(\{\sigma_i : i \in \mathcal{Q}\}) \rightarrow \sigma$ aggregates signature shares to the single signature σ of the message m under the key $\text{pk}_\wedge$. Usually, this operation is considered as a part of $\text{SigGen}(\cdot)$, but we separate it to make a further construction clearer
- $\text{SigVerify}(\text{pk}_\wedge, m, \sigma) \rightarrow \{0, 1\}$ verifies signatures and responds with 1 iff the signature is valid

$\text{SigGen}(\cdot)$ operation can consist of one or several rounds. We assume Σ satisfies **EUFCMA security** under its standard assumptions.

Definition 3.2. We define a key-agreement primitive as a set of the following algorithms:

$$\mathcal{K} = (\text{Setup}, \text{Init}, \text{Derive}, \text{Resolve}, \text{Update}),$$

where:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}_\mathcal{K}$ initializes public parameters

- $\text{Init}(\text{pp}_{\mathcal{K}}, L) \rightarrow (\text{pk}_{\perp}, \text{aux})$ initializes an OR node over the given public keys from L , producing a public auxiliary data aux . $\text{pk}_{\vee}$ is defined only after the $\text{Resolve}(\cdot)$ operation (we denote $\text{pk}_{\perp}$ as an empty/unresolved value of the public key)
- $\text{Derive}(\text{pp}_{\mathcal{K}}, \text{sk}_i, \text{aux}) \rightarrow \text{sk}_{\vee}$ allows an authorized participant $i \in \mathcal{Q}$ to derive the secret key of the node
- $\text{Resolve}(\pi_{\text{res}}, \text{sk}_i, (\text{pk}_{\perp}, \text{aux})) \rightarrow (\text{pk}_{\vee}, \text{aux})$ allows to define the public key corresponding to the shared secret of node participants. Initially, it calls $\text{Derive}(\cdot)$ to receive a node secret key and then performs a corresponding public key generation $\text{pk}_{\vee}$. Proof π_{res} is required to make sure the user is authorized to resolve the node. $\text{Resolve}(\cdot)$ might be a part of $\text{Init}(\cdot)$ procedure if the initiator controls one of the keys in the node
- $\text{Update}(\text{pp}_{\mathcal{K}}, \text{sk}_i, \text{sk}'_i, \text{aux}) \rightarrow (\text{pk}'_{\vee}, \text{aux}')$ allows to refresh the key material of the user i

We assume $\mathcal{K}$ satisfies the security under **PRF-ODH** (ART) or **KEM IND-CCA** (MLS) assumptions.

Definition 3.3. We define a non-interactive argument of knowledge protocol as a set of algorithms

$$\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$$

where:

- $\text{Setup}(1^{\lambda}, \mathcal{R}) \rightarrow (\text{vp}_{\Pi}, \text{pp}_{\Pi})$ is the setup algorithm that takes the security parameter $\lambda \in \mathbb{N}$ and relation $\mathcal{R}$ and outputs verification and proving parameters $\text{vp}_{\Pi}, \text{pp}_{\Pi}$, respectively
- $\text{Prove}(\text{pp}_{\Pi}, x, w) \rightarrow \pi$ is the proving algorithm takes prover parameters pp_{Π} , public statement x , and a witness w , and outputs the proof π
- $\text{Verify}(\text{vp}_{\Pi}, x, \pi) \rightarrow \{0, 1\}$ is the verification procedure that requires verifier parameters vp , public statement x , and proof π , and outputs 1 iff π is a valid proof

Requirements for Π are to be **complete**, **knowledge sound** and **zero-knowledge** (ideally).

3.2 Propositional logic

According to [Sip12], we define propositional logic. Boolean logic is a mathematical system built around two values, **True** and **False**, called Boolean values, which are often represented by the values 1 and 0. A Boolean function f can be defined as a formula over Boolean variables:

$$f(u_1, \dots, u_n) : \{0, 1\}^n \rightarrow \{0, 1\}$$

A Boolean formula over the variables $u_1, \dots, u_n$ consists of these variables combined using the logical operators AND ($\wedge$), NOT ($\neg$), and OR ($\vee$). Such a formula is said to be in *CNF* (*Conjunctive Normal Form*) if it is a conjunction ($\wedge$) of clauses, where each clause is a disjunction ($\vee$) of variables or their negations. More generally, a CNF formula has the form:

$$f_{\text{CNF}}(u_1, \dots, u_n) := \bigwedge_i \left(\bigvee_j v_{ij} \right),$$

where each v_{ij} is either a variable $u_{k \in [n]}$ or its negation $\neg u_k$.

Since every function can be represented by a formula that uses only the connectives $\neg, \vee, \wedge$, it follows that each such formula can be transformed into an equivalent CNF form using the following laws:

- **Associative law.** $(x \vee y) \vee z = x \vee (y \vee z), \quad (x \wedge y) \wedge z = x \wedge (y \wedge z)$
- **Distributive law.** $(x \vee y) \wedge z = (x \wedge z) \vee (y \wedge z), \quad (x \wedge y) \vee z = (x \vee z) \wedge (y \vee z)$
- **Absorption law.** $x \wedge (x \vee y) = x, \quad x \vee (x \wedge y) = x$

4 Protocol

In this section, we describe our main contribution—a *multi-party signature protocol* based on policy circuits. The high-level idea of the protocol is to apply an arbitrary logic set by some Boolean circuit for an efficiently aggregatable multi-party signature protocol (such as *BLS*[BLS01], *MuSig2*[NRS20], *MuSig-DN*[Nic+20], *MuSig-L*[BTT22]), achieving efficient “one public key verifiable” signatures.

Let’s first observe that all *n-of-n* signature protocols model a conjunction of logical propositions: *1st party has signed* **and** *2nd party has signed* **and** ... **and** *n-th party has signed*. Hence, we could efficiently emulate an AND gate by applying some multi-party signature protocol. We could also naively emulate an OR gate by applying a *continuous group key agreement protocol (CGKA)* based on either *DHE* (such as *ART*) or *KEM* (e.g. *MLS*), as every leaf in a CGKA tree could efficiently compute a root key; therefore, it could be used during signature generation to represent the condition “one of the group must sign”.

By combining these gates in the policy tree and verifiable operations, we obtain a complete construction that enables “program public keys” with arbitrary authorization logic.

4.1 BLISK Authorization Circuits

This section formalizes the authorization mechanism used in BLISK. We begin by defining the class of Boolean circuits used to express authorization policies. We then show how such circuits are compiled into cryptographic material using the primitives defined in Section 3, yielding what we call cryptographic authorization circuits. The compilation preserves the semantics of the original Boolean circuit, in that satisfying assignments correspond exactly to sets of participants capable of producing valid signatures.

Let $\mathcal{U}$ be a finite set of participants. An authorization policy is represented as a monotone Boolean circuit over variables included in $\mathcal{U}$.

Definition 4.1 (Monotone Boolean Circuit). *A monotone Boolean circuit is a tuple*

$$\mathcal{C} = (\mathcal{V}, \mathcal{E}, r, \ell, \alpha), \quad \text{where}$$

- $(\mathcal{V}, \mathcal{E})$ is finite directed acyclic graph with nodes $v \in \mathcal{V}$ and edges $e \in \mathcal{E}$
- $r = v_{\max} \in \mathcal{V}$ is a circuit root (output node)
- $\ell : \mathcal{V} \rightarrow \{\text{in}, \wedge, \vee\}$ the function that labels each node with a gate type (in is an input wire, $\wedge$ and $\vee$ are AND and OR gates, respectively)
- $\alpha : \{v \in \mathcal{V} : \ell(v) = \text{in}\} \rightarrow \mathcal{U}$ the function that labels each input node v with a distinct participant

The **monotone** means the circuit contains no negation gates.

We require that every policy circuit must be presented in *conjunctive normal form* (CNF) so that the root node contains the only $\wedge$ gate (see Figure 1). Note that this is true for any proposition from $\{\wedge, \vee\}$ gates, as it could be transformed into CNF with the distributive law.

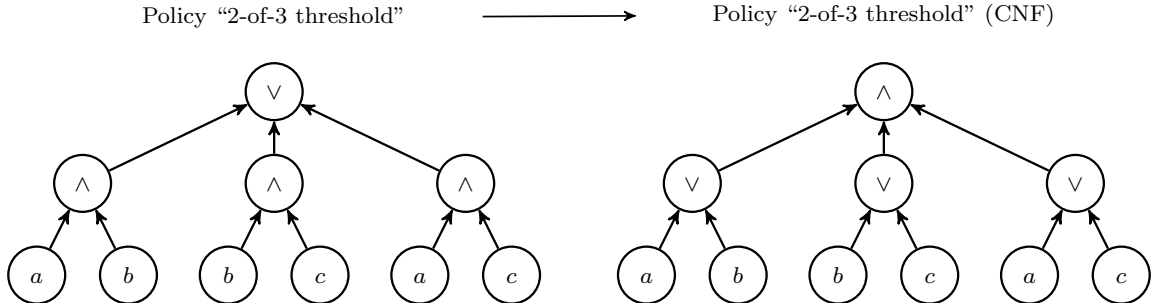


Figure 1: An illustration of transforming a general Boolean formula for a policy “2-of-3” threshold into CNF form.

Definition 4.2. We call a monotone boolean function f a threshold boolean function if it has the form:

$$f(x) = \bigvee_{S \subseteq V, |S|=k} \left(\bigwedge_{x \in S} x \right)$$

Theorem 4.1. *CNF of Threshold Functions.* Let $V = \{x_1, x_2, \dots, x_n\}$ be a set of n Boolean variables. Let f be the Boolean function defined as the disjunction of all conjunctions of size k (where $1 \leq k \leq n$):

$$f(x) = \bigvee_{S \subseteq V, |S|=k} \left(\bigwedge_{x \in S} x \right)$$

Then, the conjunctive normal form of f is the conjunction of all disjunctions of size $m = n - k + 1$:

$$f(x) \equiv \bigwedge_{Q \subseteq V, |Q|=n-k+1} \left(\bigvee_{x \in Q} x \right)$$

Proof. First, we establish the semantic meaning of the function f . A conjunction of variables is true if and only if every variable in the conjunction is true. Thus, the disjunction of all such conjunctions of size k evaluates to true if and only if there exists at least one subset of k variables that are all true. Consequently, f is the threshold function which is true if and only if the number of true variables is at least k . Let $T \subseteq V$ be the set of variables assigned the value *true*. We have $f(x) = 1 \iff |T| \geq k$.

Let $g(x)$ denote the proposed CNF expression:

$$g(x) = \bigwedge_{Q \subseteq V, |Q|=n-k+1} \left(\bigvee_{x \in Q} x \right)$$

For $g(x)$ to be true, every clause (disjunction) must be satisfied. A clause Q is satisfied if $Q \cap T \neq \emptyset$ (i.e., the clause contains at least one true variable). We now prove that $f(x) \iff g(x)$.

Direction ($\Rightarrow$): Assume $f(x) = 1$. Then $|T| \geq k$. The set of false variables, $V \setminus T$, has size $|V| - |T| \leq n - k$. Consider any arbitrary clause Q of size $n - k + 1$. By the Pigeonhole Principle, it is impossible for all variables in Q to be false, because the size of Q exceeds the total count of false variables ($n - k + 1 > n - k$). Therefore, every clause Q must contain at least one true variable. Thus $g(x) = 1$.

Direction ($\Leftarrow$): Assume $g(x) = 1$. This implies that every subset of V of size $n - k + 1$ contains at least one true variable. Suppose, for the sake of contradiction, that $f(x) = 0$. This would imply $|T| < k$, and consequently, the number of false variables is $|V \setminus T| \geq n - k + 1$. Since there are at least $n - k + 1$ false variables, we can construct a specific clause Q^* consisting entirely of false variables. If such a clause exists, the disjunction $\bigvee_{x \in Q^*} x$ evaluates to false, rendering the entire conjunction $g(x)$ false. This contradicts the assumption that $g(x) = 1$. Therefore, we must have $|T| \geq k$, and thus $f(x) = 1$. $\square$

Definition 4.3 (Circuit Evaluation). Given a subset $\mathcal{Q} \subseteq \mathcal{U}$, the evaluation function $\text{Eval}(\mathcal{C}, \mathcal{Q}) \rightarrow \{\text{True}, \text{False}\}$ on the circuit $\mathcal{C}$ on $\mathcal{Q}$ is defined as follows:

1. an input node labeled by u evaluates to **True** iff $u \in \mathcal{Q}$
2. a conjunction node evaluates to **True** iff all of its children evaluate to **True**
3. a disjunction node evaluates to **True** iff at least one of its children evaluates to **True**

We say that $\mathcal{Q}$ satisfies the circuit $\mathcal{C}$ if $\text{Eval}(\mathcal{C}, \mathcal{Q}) = \text{True}$.

This class of circuits captures monotone access structures and is expressive enough to represent threshold, hierarchical, and general Boolean authorization policies.

4.1.1 Compiling the Circuit to the Key

We now describe how a monotone Boolean circuit is compiled into cryptographic material. The compilation is parameterized by the primitives introduced in Section 3—a multisignature scheme with an effective aggregation Σ , a key-agreement primitive $\mathcal{K}$, and a non-interactive argument of a knowledge system Π .

Each participant $u \in \mathcal{U}$ holds a long-term signature key pair $(\text{sk}_u, \text{pk}_u) \leftarrow \Sigma.\text{KeyGen}$.

Definition 4.4 (Authorization Circuit Compilation). *Let $\mathcal{C} = (\mathcal{V}, \mathcal{E}, r, \ell, \alpha)$ be a monotone Boolean circuit and $\mathcal{Q}$ be a signer subset, such as $\text{Eval}(\mathcal{C}, \mathcal{Q}) = \text{True}$. We define the function $\text{Compile}(\text{pps}, \mathcal{C}, \mathcal{Q}) \rightarrow \mathcal{C}_{\text{BLISK}} = \{\text{pk}_v, \text{aux}_v\}^{|\mathcal{V}|}$ which matches each v from the Boolean circuit to the tuple $(\text{pk}_v, \text{aux}_v)$, where pk_v is a public key assigned for node v and aux_v is an auxiliary data. Note that the compilation depends on the gate type of v .*

Auxiliary data for the node v consists of the tuple $\text{aux}_v = (Ch(v), \ell(v))$, where $Ch(v)$ is the set of children nodes for the node v and $\ell(v)$ duplicates the gate type from a Boolean circuit.

Gate-Level Compilation. We define the compilation process and a compiled state for each node type:

Input nodes. If v is an input node ($\ell(v) = \text{in}$) labeled by participant u , then:

- $\text{pk}_v \leftarrow \text{pk}_u$
- $\text{aux}_v \leftarrow (\perp, \text{in})$
- $\text{sk}_v \leftarrow \text{sk}_u$

Thus, controlling an input node is equivalent to possessing the participant’s signing key.

Conjunction gates. If v is a conjunction gate with children set $Ch(v) \subset \mathcal{V}$, then:

- $\text{pk}_v \leftarrow \Sigma.\text{KeyAgg}(Ch(v))$
- $\text{aux}_v \leftarrow (Ch(v), \wedge)$
- sk_v exists only if all corresponding $\text{sk}_{i \in Ch(v)}$ are available, reflecting the semantics of $\wedge$

Disjunction gates. If v is a disjunction gate with children $Ch(v) \subset \mathcal{V}$, the compilation uses $\mathcal{K}$ to realize logical OR. Note that all OR gates are fan-2 (any fan- n OR gate could be considered a composition of fan-2 gates by associativity) and must be resolved interactively by one of their children during the initial phase, analogous to how the root key of an asynchronous ratchet tree (ART) is computed.

- $(\text{pk}_\perp, \text{aux}_v) \leftarrow \mathcal{K}.\text{Init}(\text{pp}_{\mathcal{K}}, Ch(v))$:
 pk_v isn’t defined yet
 $\text{aux}_v \leftarrow (Ch(v), \vee)$
- $\text{sk}_v \leftarrow \mathcal{K}.\text{Derive}(\text{pp}_{\mathcal{K}}, \text{sk}_{i \in Ch(v)}, \text{aux}_v)$
- $(\text{pk}_v, \text{aux}_v) \leftarrow \mathcal{K}.\text{Resolve}(\pi_{\text{res}_i}, \text{sk}_i, (\text{pk}_\perp, \text{aux}_v))$, where $i \in Ch(v)$

Proof-Carrying Compilation. BLISK requires that the compilation of authorization circuits be publicly verifiable. In particular, disjunction gates introduce nontrivial structure that must be validated. For each disjunction gate $\vee$, we define an authorization relation $\mathcal{R}_\vee$:

$$\mathcal{R}_\vee = \left\{ \begin{array}{l} (\text{pk}_v, \text{aux}_v); \\ (\text{sk}, \text{pk}) \end{array} \middle| \begin{array}{l} \pi_{\text{res}_i} \leftarrow \text{PoK}(\text{pk} \in Ch(v)) \\ \text{sk}_v \leftarrow \mathcal{K}.\text{Derive}(\text{pp}_{\mathcal{K}}, \text{sk}, \text{aux}_v) \\ (\text{pk}_v, \text{aux}_v) \leftarrow \mathcal{K}.\text{Resolve}(\pi_{\text{res}_i}, \text{sk}, (\text{pk}_\perp, \text{aux}_v)) \end{array} \right\}$$

We don’t include the $\text{Init}(\cdot)$ operation in the relation, because it naturally is performed by the group coordinator (and every participant can check the authorization circuit publicly).

So we make a setup for the proof system in advance: $(\mathbf{vp}_\Pi, \mathbf{pp}_\Pi) \leftarrow \text{Setup}(1^\lambda, \mathcal{R}_\vee)$. The witness of the proof $\pi_{\vee_v} \leftarrow \Pi.\text{Prove}(\mathbf{pp}_\Pi, (\mathbf{pk}_v, \mathbf{aux}_v), \mathbf{sk}_{v \in \text{Ch}(v)})$ attests that $(\mathbf{pk}_v, \mathbf{aux}_v)$ was produced by a valid execution of $\mathcal{K}.\text{Resolve}(\cdot)$ and a secret key $\mathbf{sk}_v$ is derivable from at least one child secret key using $\mathcal{K}.\text{Derive}(\cdot)$. Proof is verified by the instance $\Pi.\text{Verify}(\cdot)$. A cryptographic authorization circuit is valid if all required proofs verify.

Definition 4.5 (Cryptographic Satisfiability of Circuit). *A set of participants $\mathcal{Q} \subseteq \mathcal{U}$ cryptographically satisfies the compiled circuit if, for every node $v \in \mathcal{V}$ that evaluates $\text{Eval}(\mathcal{C}, \mathcal{Q}) = \text{True}$, the participants in $\mathcal{Q}$ can jointly obtain the corresponding secret key $\mathbf{sk}_v$ (obtain can mean not only the direct access to the key, but an ability to generate the signature shard with together with all required shares can satisfy the common public key).*

In particular, cryptographic satisfiability of the root node r implies possession of $\mathbf{sk}_r$. Moreover, if $\mathcal{Q}$ satisfies $\mathcal{C}$, then $\mathcal{Q}$ can produce a valid signature under the single verification key $\mathbf{pk}_r$ associated with the root node, using the signature scheme Σ . So, this approach establishes a correspondence between Boolean authorization logic and cryptographic realizability: the policy's Boolean satisfiability is equivalent to cryptographic key control.

Sequential Compilation of Authorization circuits In addition to structural correctness, BLISK requires that authorization circuits be resolved sequentially by participants, ensuring that every compiled node reflects explicit approval by the participants responsible for that node.

A sequential compilation order is a total order $\prec$ on $\mathcal{V}$ such that:

1. $\prec$ is a topological order of $(\mathcal{V}, \mathcal{E})$, i.e., for every edge $(x \rightarrow v) \in \mathcal{E}$, we have $x \prec v$
2. all input nodes precede all internal nodes

For each node $v \in \mathcal{V}$, define the set of responsible participants $\text{Resp}(v) \subseteq \mathcal{Q}$ as follows:

- If $\ell(v) = \text{in}$ and $\alpha(v) = u$, then $\text{Resp}(v) = \{u\}$.
- If $\ell(v) = \wedge$, then

$$\text{Resp}(v) = \bigcup_{x \in \text{Ch}(v)} \text{Resp}(x).$$

- If $\ell(v) = \vee$, then $\text{Resp}(v) = \text{Resp}(x)$ for some child $x \in \text{Ch}(v)$.

For disjunction gates, the choice of x is existential: it suffices that at least one child determines the responsibility set.

Definition 4.6 (Sequential Compilation/Resolution Protocol). *Let $\prec$ be a sequential compilation order. The sequential compilation protocol proceeds in rounds indexed by nodes $v \in \mathcal{V}$ in increasing order under $\prec$. In round v , the following steps are executed:*

1. Availability. *The compiled public states $(\mathbf{pk}_x, \mathbf{aux}_x)$ of all children $x \in \text{Ch}(v)$ are available.*
2. Resolution. *The participants in $\text{Resp}(v)$ jointly execute the compilation algorithm corresponding to $\ell(v)$, producing $(\mathbf{pk}_v, \mathbf{aux}_v)$.*
3. Proof generation. *The participants in $\text{Resp}(v)$ generate a proof π_{res} attesting to the correct resolution of node v with respect to its children.*
4. Publication. *The tuple $(\mathbf{pk}_v, \mathbf{aux}_v, \pi_v)$ is published and becomes immutable input for all subsequent rounds.*

4.2 BLISK Multisignature

Definition 4.7. We define a BLISK multisignature as a set of algorithms:

$$\Sigma_{\text{BLISK}} = \{\text{Setup}, \text{KeyGen}, \text{PolicyCircuitInit}, \text{AcceptPolicy}, \text{KeyAgg}, \text{SigGen}, \text{SigAgg}, \text{SigVerify}, \text{KeyUpdate}\}$$

Parameters setup. $\text{Setup}_{\text{BLISK}}^{\Sigma, \mathcal{K}, \Pi}(1^\lambda) := (\Sigma.\text{Setup}(1^\lambda), \mathcal{K}.\text{Setup}(1^\lambda), \Pi.\text{Setup}(1^\lambda)) \rightarrow (\text{pps}, \text{pp}_\Sigma, \text{pp}_\mathcal{K}, \text{pp}_\Pi)$. On input 1^λ , the setup algorithm initializes the global parameters pps for the system, signature primitive Σ , the key-agreement primitive $\mathcal{K}$, and the zero-knowledge proof system Π by calling the corresponding protocols' setup instances. Since we describe the general framework, we don't define what these parameters can be. But you can find an example of parameters for the specific construction described in Appendix A.

Key generation. $\text{KeyGen}(1^\lambda) \rightarrow (\text{sk}, \text{pk})$. Each signer in $\mathcal{U}$ generates a random secret value sk and derives a corresponding public key pk .

Policy Circuit Initialization. $\text{PolicyCircuitInit} : (\mathcal{Q}, \mathcal{C}) \rightarrow \mathcal{C}_{\text{BLISK}}$. Given a set of participants' public keys $\mathcal{Q} \subseteq \{\text{pk}_u : u \in \mathcal{U}\}$ and a monotone Boolean authorization circuit $\mathcal{C} = (\mathcal{V}, \mathcal{E}, r, \ell, \alpha)$, the initialization algorithm compiles a policy circuit $\mathcal{C}_{\text{BLISK}} = \{(\text{pk}_{v/\perp}, \text{aux}_v)\}_{v \in \mathcal{V}} \leftarrow \text{Compile}(\text{pps}, \mathcal{C}, \mathcal{Q})$ according to compilation rules. The output $\mathcal{C}_{\text{BLISK}}$ is an inactive policy circuit: the root public key pk_r as well as keys for $\vee$ gates are not determined. Appropriate keys may be derived only later through the sequential resolution and acceptance procedures.

Policy acceptance. $\text{AcceptPolicy}(\text{sk}_u, \mathcal{C}_{\text{BLISK}}) \rightarrow (\text{branch}_u, \pi_u)$. Given a participant secret key sk_u and a public policy circuit $\mathcal{C}_{\text{BLISK}} = \{(\text{pk}_{v/\perp}, \text{aux}_v)\}_{v \in \mathcal{V}}$, the policy acceptance algorithm allows participant u to activate those nodes of the policy circuit that correspond to input nodes labeled by u and their induced parent nodes. An algorithm output consists of all derived secret material and proofs required to justify participant u 's contribution to the policy tree. All proofs are publicly verifiable using $\Pi.\text{Verify}$. Policy acceptance is a sequential process as described in Section 4.1. When all required proofs are provided, we know that the circuit $\mathcal{C}_{\text{BLISK}}$ is active, so pk_r is defined.

Key Aggregation. $\text{KeyAgg}_{\text{BLISK}}(\mathcal{C}_{\text{BLISK}}) \rightarrow \text{pk}_{\text{agg}}$. Let $\mathcal{C}_{\text{BLISK}}$ be an active public policy circuit. Intuitively, each $v \in \mathcal{V}_{\text{max}}^\vee$ represents a maximal authorized branch contributing signing authority under the policy. The aggregated public key is then computed as

$$\text{pk}_{\text{agg}} \leftarrow \Sigma.\text{KeyAgg}(\{\text{pk}_v : v \in \mathcal{V}_{\text{max}}^\vee\}).$$

Only public keys corresponding to nodes in $\mathcal{V}_{\text{max}}^\vee$ are included in the aggregation. No public keys from internal nodes below these disjunction gates or from unsatisfied branches contribute to pk_{agg} .

Signature generation. $\text{SigGen}(\text{sk}_i, \text{pk}_{\text{agg}}, m) \rightarrow (\sigma_i)$. The signature generation algorithm is literally $\Sigma.\text{SigGen}$ run on the clauses keys $\mathcal{V}_{\text{max}}^\vee$.

Signature aggregation. $\text{SigAgg}(\{\sigma_i\}) \rightarrow \sigma$ aggregates signature shares to the single signature σ of the message m under the key pk_r .

Signature verification. $\text{SigVerify}(\text{pk}_r, m, \sigma) \rightarrow \{0, 1\}$. $\text{SigVerify}(\cdot)$ is a regular instance of $\Sigma.\text{Verify}$ which responds with 1 in the case of the correctness of the signature σ up to the public key pk_{agg} and a message m , and 0 otherwise.

Key update. $\text{KeyUpdate}(\text{sk}_u, \text{sk}'_u, \mathcal{C}_{\text{BLISK}}) \rightarrow (\mathcal{C}'_{\text{BLISK}}, \pi_{\text{upd}})$. Given a participant's old secret key sk_u , a refreshed secret key sk'_u , and a public policy circuit $\mathcal{C}_{\text{BLISK}}$, the key update algorithm allows participant u to update all policy circuit nodes whose secret keys depend on sk_u . The output $\mathcal{C}'_{\text{BLISK}}$ consists of the updated public states $(\text{pk}'_v, \text{aux}'_v)$ and corresponding proofs π_{upd} for all updated nodes.

5 Discussion and applications

BLISK's major move is to compile authorization logic into a single verification key without changing the underlying signature verification algorithm. That seemingly small design choice has significant consequences for real applications: you can retain the "verify one key, one signature" interface while shifting expressiveness and policy management off-chain. Let's discuss the main advantages, disadvantages, and perspectives of BLISK.

Expressiveness VS “One-Key” Verification. BLISK targets the gap between what policies look like in practice and what classic threshold signatures can express. The model is a monotone Boolean circuit over participants (AND/OR only), which is broad enough to encode many real policies (role-based approvals, mandatory participants, hierarchical sign-offs, “either-of-these plus all-of-those,” etc.).

At the same time, a BLISK’s real-world behavior is often dominated by the AND/OR protocol properties. For example, we show below how 1-round signature protocols can significantly improve the practical usage of BLISK. At the same time, several features (membership dynamics, coordinator assumptions, update costs, liveness) must be considered when selecting primitives.

Property	TSS	LSSS	MPC	BLISK
Policy Expressiveness	k -of- n	Monotone	Any	Monotone CFN
Signature Size	$\mathcal{O}(1)$	$\mathcal{O}(\text{Access structure})$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Verification Cost	$\mathcal{O}(1)$	$\mathcal{O}(\text{Access structure})$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Key Generation	DKG required	Share distribution	DKG required	No DKG
Key Update	Full re-DKG	Re-share	Full re-DKG	Non-interactive, per participant

Table 1: Comparison table

CNF Matters. A strong structural constraint is that policies must be represented in CNF with a single $\wedge$ at the root. That’s a workable normalization, but it has an important practical implication: CNF conversion can blow up policy size in the worst (specifically, a policy with k clauses of size m in DNF may yield up to m^k clauses in CNF). Even if many realistic policies remain manageable, a “policy compiler” that automatically rewrites arbitrary formulas into CNF needs:

- Human-friendly policy language (like S-expressions, but with additional tools like policy size estimates, warnings, etc).
- Some additional tools that allow the reduction and optimization of the initial written policy, together with the formal verification that the resulting policy corresponds to the initial one.

Proof-Carrying Compilation. A proof system choice (soundness, setup assumptions, verification cost) becomes a key engineering and trust decision. In the paper, we abstract this as Π , but deployment, associated costs, and assumptions will depend on the selected system.

Additional Policy Logic. The next line of research can focus on improving BLISK under conditions that monotone Boolean circuits can’t express. For example, combining BLISK with adaptor signatures and DL contracts, HTLC contracts, etc., can expand the range of real-world use cases for BLISK.

Bitcoin Integration. BLISK protocols integrates naturally with Bitcoin Taproot(BIP 341) [WPS21] structure

$$Q = P + [\mathcal{H}(P, m)]G, \quad \text{where}$$

- **Key Path:** The aggregated root key of the policy circuit becomes the internal key P .
 - It allows spending coins with a single signature and a single public key on-chain, provided the policy is satisfied.
 - It hides the policy’s complexity and the specific signers involved.
- **Script Path:** Can remain as a backup or for alternative spending conditions (e.g., with hash locks which aren’t supported by BLISK yet) via the Merkle tree.

Ethereum Integration. We also see several options to apply BLISK to the Ethereum infrastructure:

- **BLS Instantiation** ($\Sigma_{\text{BLISK}}^{\text{BLS+ART}}$):
 - Unlike Schnorr (MuSig2), BLS signatures are non-interactive (1 round).

- It allows asynchronous signing: parties sign their resolutions independently without coordinating rounds.
- Crucial for distributed setups (e.g., DAOs, DVT) where liveness is a challenge.
- **Gas Efficiency via Aggregation.** Using BLISK, we can resolve the policy off-chain. The EVM only verifies a single aggregated signature against a single root public key (now, on-chain multisigs (e.g., Gnosis Safe) execute complex logic inside the EVM, consuming gas for every signer and check). Thus, BLISK reduces the calldata size and execution cost to $O(1)$, regardless of policy complexity.
- **Account Abstraction (ERC-4337).** The aggregated root key of a Policy Tree can serve as the validation authority for an ERC-4337 [But+21] compliant smart account. The validation phase of the user operation would consist of a single signature check. This structure preserves privacy by keeping the internal governance rules (the specific “AND/OR” gates and signer identities) hidden from the public ledger, revealing only that a valid authorization was produced.

PQ Future. As we mentioned, BLISK’s security depends on the underlying primitives. We present the multisignature instance in the appendix, which relies on the “hard” ECDL problem. But at the same time, underlying constructions can rely on problems that are still hard for a quantum computer. For instance, we could combine MuSig-L[BTT22], PQXDH [FG24], and LaBRADOR [BS22] and build a PQ BLISK instance. But we also need to consider the practicality of the mentioned example.

6 Implementation

We have implemented our policy tree protocol instantiated with *MuSig2* as a core multi-signature protocol and ART as a tree-based CGKA construction in *Rust*². In that implementation, we presented *the policy compiler* for policies presented as *S-expressions*. Our compiler supports automatic conversion of an input policy to minimal *CNF* form, thus it permits any policy written using AND, OR gates, for example, a 3-of-4 threshold policy with the signers set $\mathcal{S} = \{A, B, C, D\}$ could be written as:

```
(policy threshold_3_of_4_circuit
(or
  (and A B C)
  (and A B D)
  (and A C D)
  (and B C D)))
```

6.1 Benchmarks

The tests of BLISK on different examples of policies show the following result (Table 2):

Table 2: Performance Metrics of BLISK for Policy Expressions

Policy Expression	Parse	KeyGen	Compile	Resolve	Sign Init	Sign	Verify	Total
and A1 A2	26.16 μ s	393.07 μ s	463.71 μ s	7.20 μ s	1.43 ms	2.89 ms	370.37 μ s	5.58 ms
and A1 A2 A3	25.76 μ s	573.11 μ s	693.87 μ s	7.29 μ s	2.81 ms	5.81 ms	438.20 μ s	10.36 ms
and A1 A2 A3 A4 A5	45.07 μ s	1.52 ms	1.65 ms	14.45 μ s	16.91 ms	16.97 ms	539.51 μ s	37.65 ms
and A1 A2 A3 ... A10	52.51 μ s	2.82 ms	3.46 ms	24.27 μ s	57.51 ms	63.43 ms	528.61 μ s	128.43 ms
and A1 A2 A3 ... A15	47.31 μ s	3.52 ms	4.68 ms	22.13 μ s	111.19 ms	125.69 ms	514.40 μ s	246.25 ms
and (and (and A1 A2) (and A3 A4)) (and (and A5 A6) (and A7 A8))	25.65 μ s	1.41 ms	1.72 ms	8.69 μ s	26.12 ms	26.48 ms	377.24 μ s	56.14 ms
and (or A1 A2) (or A3 A4)	25.83 μ s	750.30 μ s	47.59 μ s	3.37 ms	2.74 ms	2.90 ms	383.12 μ s	10.22 ms

Continued on next page

²<https://github.com/zero-art-rs/blisk>

Table 2: Performance Metrics of BLISK for Policy Expressions (continued)

Policy Expression	Parsing	KeyGen	Comp.	Resol.	Sign Init.	Sign.	Verif.	Total
and (or A1 A2) (or A3 A4) (or A5 A6)	49.65 μ s	1.33 ms	56.91 μ s	9.58 ms	5.31 ms	9.40 ms	409.15 μ s	26.14 ms
and (or A1 A2) (or A3 A4) (or A5 A6) (or A7 A8) (or A9 A10)	62.17 μ s	2.35 ms	80.14 μ s	23.49 ms	11.52 ms	21.74 ms	433.34 μ s	60.18 ms
and (or A1 (and A2 (or A3 A4))) (or (and A5 A6 A7) A8) (and A9 A10)	36.03 μ s	2.37 ms	254.03 μ s	24.84 ms	26.96 ms	21.42 ms	415.42 μ s	76.30 ms
threshold 2 A1 A2 A3	30.90 μ s	627.85 μ s	106.17 μ s	4.56 ms	6.60 ms	4.04 ms	432.52 μ s	16.40 ms
threshold 2 A1 A2 A3 A4 A5	36.73 μ s	1.17 ms	202.77 μ s	27.81 ms	17.38 ms	6.56 ms	420.94 μ s	53.59 ms
threshold 3 A1 A2 A3 A4 A5	35.23 μ s	1.06 ms	243.62 μ s	41.78 ms	37.60 ms	20.52 ms	459.43 μ s	101.70 ms
threshold 4 A1 A2 A3 A4 A5	50.36 μ s	1.67 ms	251.41 μ s	33.41 ms	37.45 ms	26.28 ms	466.21 μ s	99.58 ms
threshold 3 A1 ... A10	40.85 μ s	2.05 ms	2.66 ms	886.83 ms	504.82 ms	120.70 ms	594.75 μ s	1517.70 ms
threshold 5 A1 ... A10	30.47 μ s	1.75 ms	10.73 ms	2.10 s	2.26 s	0.63 s	365.14 μ s	5.00 s
threshold 7 A1 ... A10	77.21 μ s	4.21 ms	16.96 ms	2.20 s	3.47 s	1.39 s	513.00 μ s	7.08 s
threshold 11 A1 ... A13	31.00 μ s	2.22 ms	8.48 ms	1.98 s	3.60 s	1.65 s	434.32 μ s	7.24 s
threshold 3 A1 ... A15	31.40 μ s	2.57 ms	10.51 ms	3.70 s	1.13 s	0.20 s	355.04 μ s	5.04 s
threshold 12 A1 ... A15	27.44 μ s	2.28 ms	128.97 ms	14.45 s	44.20 s	17.42 s	319.32 μ s	76.20 s
threshold 13 A1 ... A15	27.58 μ s	2.33 ms	15.86 ms	3.30 s	6.80 s	2.96 s	303.10 μ s	13.08 s
threshold 17 A1 ... A20	33.16 μ s	3.44 ms	1.52 s	117.19 s	635.57 s	422.32 s	392.49 μ s	1176.69 s
and A1 (threshold 2 A2 A3 A4 A5) A6	26.97 μ s	1.05 ms	248.39 μ s	13.54 ms	17.27 ms	11.37 ms	368.60 μ s	43.87 ms
and (threshold 2 A1 A2 A3) (threshold 2 A4 A5 A6)	87.39 μ s	2.12 ms	771.19 μ s	33.31 ms	28.74 ms	20.52 ms	590.58 μ s	86.14 ms
or (and A1 (or A2 (threshold 2 (or A3 A4) A5 A6 A7 A8 A9 (and A10 A11))))	29.07 μ s	1.91 ms	8.77 ms	170.62 ms	51.75 ms	16.00 ms	356.25 μ s	249.44 ms
or (and A1 (and (threshold 3 A2 ... A7) (thresh 2 (or A8 A9) A10 ... A14 (and A15 A16))))	31.85 μ s	2.78 ms	10.43 ms	387.82 ms	225.82 ms	82.09 ms	368.72 μ s	709.34 ms
threshold 2 A1 (threshold 2 A2 (threshold 2 A3 (threshold 2 A4 A5 A6)))	29.67 μ s	1.39 ms	271.42 μ s	11.63 ms	20.92 ms	17.09 ms	437.09 μ s	51.77 ms
threshold 2 (threshold 2 A1 A2 A3) (threshold 2 A4 A5 A6) (threshold 2 A7 A8 A9) (threshold 2 A10 A11 A12)	28.34 μ s	2.15 ms	70.33 ms	1.46 s	26.96 ms	1004.54 ms	357.44 μ s	2.56 s

References

- [Alw+20] Joël Alwen et al. *Asynchronous Ratcheting Trees for Secure Group Messaging*. Introduces ART, a tree-based ratcheting construction for asynchronous group communication. 2020. URL: <https://eprint.iacr.org/2020/1281>.
- [Bar+19] Richard Barnes et al. *Tree-Based Group Key Agreement*. Early formalization of TreeKEM used in MLS. 2019. URL: <https://eprint.iacr.org/2019/1088>.
- [Bei25] Amos Beimel. *Secret-Sharing Schemes for General Access Structures: An Introduction*. Cryptology ePrint Archive, Paper 2025/518. 2025. URL: <https://eprint.iacr.org/2025/518>.
- [BS22] Ward Beullens and Gregor Seiler. *LaBRADOR: Compact Proofs for R1CS from Module-SIS*. Cryptology ePrint Archive, Paper 2022/1341. 2022. URL: <https://eprint.iacr.org/2022/1341>.

- [Bit09] Bitcoin Core Developers. *OP_CHECKMULTISIG — Bitcoin Script Opcode*. Bitcoin scripting opcode documentation. 2009. URL: <https://developer.bitcoin.org/reference/transactions.html#op-checkmultisig>.
- [Bol02] Alexandra Boldyreva. “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme”. In: *Public Key Cryptography — PKC 2003*. Ed. by Yvo G. Desmedt. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 31–46.
- [BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. “Short Signatures from the Weil Pairing”. In: *Advances in Cryptology — ASIACRYPT 2001*. Ed. by Colin Boyd. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 514–532. ISBN: 978-3-540-45682-7.
- [BTT22] Cecilia Boschini, Akira Takahashi, and Mehdi Tibouchi. “MuSig-L: Lattice-Based Multisignature with Single-Round Online Phase”. In: *Advances in Cryptology – CRYPTO 2022*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Cham: Springer Nature Switzerland, 2022, pp. 276–305. ISBN: 978-3-031-15979-4.
- [Bün+18] Benedikt Bünz et al. *Bulletproofs: Short Proofs for Confidential Transactions and More*. Non-interactive zero-knowledge proofs without trusted setup. 2018. URL: <https://eprint.iacr.org/2017/1066>.
- [But+21] Vitalik Buterin et al. *ERC-4337: Account Abstraction Using Alt Mempool*. <https://eips.ethereum.org/EIPS/eip-4337>. Ethereum Improvement Proposal. Accessed: 2025-12-26. 2021.
- [DR23] Sourav Das and Ling Ren. *Adaptively Secure BLS Threshold Signatures from DDH and co-CDH*. Cryptology ePrint Archive, Paper 2023/1553. 2023. URL: <https://eprint.iacr.org/2023/1553>.
- [EKR18] David Evans, Vladimir Kolesnikov, and Mike Rosulek. “A Pragmatic Introduction to Secure Multi-Party Computation”. In: *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246. ISSN: 2474-1558. DOI: 10.1561/33000000019. URL: <http://dx.doi.org/10.1561/33000000019>.
- [FG24] Rune Fiedler and Felix Günther. *Security Analysis of Signal’s PQXDH Handshake*. Cryptology ePrint Archive, Paper 2024/702. 2024. DOI: 10.1007/978-3-031-91823-0_5. URL: <https://eprint.iacr.org/2024/702>.
- [GG19] Rosario Gennaro and Steven Goldfeder. *Fast Multiparty Threshold ECDSA with Fast Trustless Setup*. Cryptology ePrint Archive, Paper 2019/114. 2019. URL: <https://eprint.iacr.org/2019/114>.
- [GGN16] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. “Threshold-Optimal DSA/ECDSA Signatures and an Application to Bitcoin Wallet Security”. In: *Applied Cryptography and Network Security*. Ed. by Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider. Cham: Springer International Publishing, 2016, pp. 156–174. ISBN: 978-3-319-39555-5.
- [Gen+99] Rosario Gennaro et al. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *Journal of Cryptology* 20 (1999), pp. 51–83. URL: <https://api.semanticscholar.org/CorpusID:3331212>.
- [Gen+07] Rosario Gennaro et al. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *Journal of Cryptology* 20.1 (2007), pp. 51–83. DOI: 10.1007/s00145-006-0347-3. URL: <https://doi.org/10.1007/s00145-006-0347-3>.
- [Gno18] Gnosis Ltd. *Gnosis Safe: A Secure Multisignature Wallet*. Now known as Safe. 2018. URL: <https://gnosis-safe.io>.
- [Hai+18] Iftach Haitner et al. *Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody*. Cryptology ePrint Archive, Paper 2018/987. 2018. DOI: 10.1145/3243734.3243788. URL: <https://eprint.iacr.org/2018/987>.
- [Hru+25] Yevhen Hrubian et al. *0-ART. Asynchronous and Verifiable Group Management for Decentralized Applications*. Cryptology ePrint Archive, Paper 2025/1874. 2025. URL: <https://eprint.iacr.org/2025/1874>.

- [kAN20] koe, Kurt M. Alonso, and Sarang Noether. *Zero to Monero: A technical guide to a private digital currency; for beginners, amateurs, and experts*. Second. Public Domain, 2020. URL: <https://www.getmonero.org/library/Zero-to-Monero-2-0-0.pdf>.
- [Lin20] Yehuda Lindell. *Secure Multiparty Computation (MPC)*. Cryptology ePrint Archive, Paper 2020/300. 2020. DOI: 10.1145/3387108. URL: <https://eprint.iacr.org/2020/300>.
- [NRS20] Jonas Nick, Tim Ruffing, and Yannick Seurin. *MuSig2: Simple Two-Round Schnorr Multi-Signatures*. Cryptology ePrint Archive, Paper 2020/1261. 2020. DOI: 10.1007/978-3-030-84242-0.8. URL: <https://eprint.iacr.org/2020/1261>.
- [Nic+20] Jonas Nick et al. *MuSig-DN: Schnorr Multi-Signatures with Verifiably Deterministic Nonces*. Cryptology ePrint Archive, Paper 2020/1057. 2020. DOI: 10.1145/3372297.3417236. URL: <https://eprint.iacr.org/2020/1057>.
- [24] *sips/sips/sip-025/sip-025-iterating-towards-weighted-schnorr-threshold-signatures.md at main · stacksgov/sips — github.com*. https://github.com/stacksgov/sips/blob/main/sips/sip-025/sip-025-iterating-towards-weighted-schnorr-threshold-signatures.md?utm_source=chatgpt.com. [Accessed 08-12-2025]. 2024.
- [Sip12] M. Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2012. ISBN: 9781133187790. URL: <https://books.google.com.ua/books?id=P3f6CAAAQBAJ>.
- [WPS21] Pieter Wuille, Andrew Poelstra, and Yannick Seurin. *BIP-341: Taproot: SegWit Version 1 Spending Rules*. Bitcoin Improvement Proposal. 2021. URL: <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki>.

A Example of BLISK signature in MuSig2 and 0-ART settings

A.1 Example

Let's consider the exact policy: "Alice is required AND (Bob OR Carol)".

We set $\Sigma_{\text{BLISK}}^{\text{MuSig2}[v=4], 0\text{-ART}[\text{BP}]}$:

$$\begin{aligned}\Sigma &\leftarrow \text{MuSig2}[v=4][\text{NRS20}] \\ \mathcal{K} &\leftarrow 0\text{-ART}[\text{Hru}+25] : \Pi \leftarrow \text{Bulletproofs}[\text{Bün}+18] \\ \mathbb{G} &\leftarrow \text{secp256k1} \\ \mathcal{H}_{\text{sig}}, \mathcal{H}_{\text{art}} &\text{— Schnorr challenge and ART key derivation hashes}\end{aligned}$$

1. Setup. We run a setup for the BLISK signature instance by

$$\text{pps} = \{\mathbb{G}, q, G, \mathcal{H}_{\text{sig}}, \mathcal{H}_{\text{art}}\} \leftarrow \Sigma_{\text{BLISK}}.\text{Setup}(1^\lambda)$$

2. Key Generation. Every user $x \in \{A, B, C\}$ generates a cryptographic key pair $(\text{sk}_x, \text{pk}_x) \leftarrow \Sigma_{\text{BLISK}}.\text{KeyGen}(1^\lambda)$ as:

$$\text{sk}_{x \in A, B, C} \xleftarrow{\$} \mathbb{F}_q, \text{pk}_x = [\text{sk}_x]G$$

3. Policy circuit initialization. The coordinator creates an inactive circuit for the policy. We have the following circuit (already in CNF):

$$A \wedge (B \vee C)$$

Circuit structure:

- Input nodes:

$$v_{x \in A, B, C} \leftarrow \text{pk}_x$$

- Internal nodes:

- $v_\vee$: OR gate over v_B and v_C
- $v_r = v_\wedge$: AND gate over v_A and $v_\vee$

Or graphically:

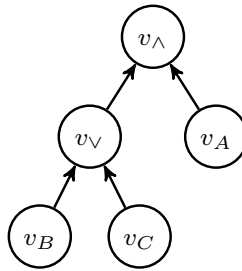


Figure 2: A tree-based schematic representation of the policy "Alice and one of Bob or Carol must sign".

For each node v the coordinator initializes a public state $\mathcal{C}_{\text{BLISK}} = (\text{pk}_v, \text{aux}_v)$

$$\begin{aligned}\text{pk}_{v_{x \in A, B, C}} &\leftarrow \text{pk}_x, \text{aux}_v = \perp \\ (\text{pk}_\perp, \text{aux}_{v_\vee}) &= (\text{pk}_\perp, \{\text{pk}_B, \text{pk}_C\}) \leftarrow 0\text{-ART}.\text{Init}(\{\mathbb{G}, q, G, \mathcal{H}_{\text{art}}\}, \{\text{pk}_B, \text{pk}_C\}) \\ \text{pk}_\wedge &\leftarrow \perp, \text{aux}_\wedge \leftarrow \perp\end{aligned}$$

At this stage $\text{pk}_{v_\vee}$ and $\text{pk}_\wedge$ are undefined.

4. Sequential Acceptance and the Circuit Compilation.

We fix a sequential order:

$$v_A \prec v_B \prec v_C \prec v_{\vee} \prec v_{\wedge}$$

Assume Bob is the party activating the $v_{\vee}$ gate (Carol's branch is symmetric). In this case each party provides $(\text{branch}_u, \pi_u) \leftarrow \text{AcceptPolicy}(\text{sk}_u, \mathcal{C}_{\text{BLISK}})$:

$$\begin{aligned} A &: (v_A, \sigma_{A,\text{acc}}) \leftarrow \text{PoK}(\text{sk}_A, c := \mathcal{H}(\mathcal{C}_{\text{BLISK}})) \\ B &: (v_B, v_{\vee}, \sigma_{B,\text{acc}}, \pi_{B,\text{acc}}) : \\ &\quad \sigma_{B,\text{acc}} \leftarrow \text{PoK}(\text{sk}_B, c := \mathcal{H}(\mathcal{C}_{\text{BLISK}})) \\ &\quad \pi_{\text{acc}} \leftarrow \Pi.\text{Prove}(\text{pp}_{\Pi}, \mathcal{R}_{v_{\vee}}, \text{sk}_B) \\ C &: (v_C, \sigma_{C,\text{acc}}) \leftarrow \text{PoK}(\text{sk}_C, c := \mathcal{H}(\mathcal{C}_{\text{BLISK}})) \end{aligned}$$

Note that the relation $\mathcal{R}_{v_{\vee}}$ in this case is:

$$\mathcal{R}_{v_{\vee}} = \left\{ (v_B, v_C), (\text{pk}_{v_{\vee}}, \text{aux}_{v_{\vee}}) \mid \begin{array}{l} \text{aux}_{v_{\vee}} = (v_B, v_C) \\ \text{sk}_{v_{\vee}} = \mathcal{H}_{\text{art}}([\text{sk}_B]v_C) \\ \text{pk}_{v_{\vee}} = [\text{sk}_{v_{\vee}}]G \end{array} \right\}$$

We can count the circuit as compiled if proofs for all participants are provided.

5. Key aggregation After every user accepted the policy, the coordinator makes a key aggregation as $\text{pk}_{\text{agg}} \leftarrow \Sigma.\text{KeyAgg}(\{\text{pk}_v : v \in \mathcal{V}_{\text{max}}^{\vee}\})$. Since we have only two children of the final $\wedge$ gate and following MuSig2 key aggregation procedure, the key is derived as:

$$\begin{aligned} a_{i \in L} &= \mathcal{H}_{\text{agg}}(L, \text{pk}_i), \quad \text{where } L = \{\text{pk}_A, \text{pk}_{\vee}\} \\ \text{pk}_{\text{agg}} &:= \sum_{i \in L} [a_i] \text{pk}_i = [a_A] \text{pk}_A + [a_{\vee}] \text{pk}_{\vee} \end{aligned}$$

6. Signature generation. Let Alice and Bob cooperate to create a signature which satisfies the policy $\mathcal{C}_{\text{BLISK}}$. Having a 2-round MuSig2 protocol with $v = 4$, the signature generation process follows:

Round 1:

for $l = 1 \dots 4$ **do**

$$r_l^{(x \in A, B)} \xleftarrow{\$} \mathbb{F}_p, \quad R_l^{(x)} = [r_l^{(x)}]G$$

$$\text{out}^{(x)} = (R_1^{(x)}, \dots, R_4^{(x)})$$

Round 2:

for $l = 1 \dots 4$ **do**

$$R_l = \sum_{x \in A, B} R_l^{(x)}$$

$$b = \mathcal{H}_{\text{non}}(\text{pk}_{\text{agg}}, (R_1, \dots, R_4), m)$$

$$R = \sum_{l=1}^4 [b^{l-1}] R_l$$

$$c = \mathcal{H}_{\text{sig}}(\text{pk}_{\text{agg}}, R, m)$$

$$s_A = (\sum_{l=1}^4 b^{l-1} \cdot r_l^{(A)}) + c \cdot a_A \cdot \text{sk}_A$$

$$s_B = (\sum_{l=1}^4 b^{l-1} \cdot r_l^{(B)}) + c \cdot a_{\vee} \cdot \mathcal{H}_{\text{art}}([\text{sk}_B] \text{pk}_C)$$

$$s = s_A + s_B$$

$$\sigma = (R, s)$$

We can see here that Alice is required here, since the knowledge of sk_A is needed to produce the signature. But at the same time, the second signature share requires the knowledge of $\mathcal{H}_{\text{art}}([\text{sk}_B] \text{pk}_C) = \mathcal{H}_{\text{art}}([\text{sk}_C] \text{pk}_B)$, which is accessible by Bob and Carol.

7. Signature Verification. The signature verification doesn't differ from the single verification. By running $\Sigma.\text{SigVerify}(\text{pk}_r, m, \sigma)$, the verifier sets the challenge

$$c = \mathcal{H}_{\text{sig}}(\text{tag}, \text{pk}_r, R, m)$$

And checks if

$$[s]G \stackrel{?}{=} R + [c]\text{pk}_r$$

8. Key Update. Let Bob rotate their secret key and make a verifiable update of the circuit $(\mathcal{C}'_{\text{BLISK}}, \pi_{\text{upd}}) \leftarrow \text{KeyUpdate}(\text{sk}_B, \text{sk}'_B, \mathcal{C}_{\text{BLISK}})$. First of all, Bob runs $(\text{sk}'_B, \text{pk}'_B) \leftarrow \Sigma_{\text{BLISK}}.\text{KeyGen}(1^\lambda)$ to receive new long-term key pair.

To update Bob's key, we need to update only the nodes in the circuit that depend on it – v_B , v_v , and v_r . To do it, Bob generates:

$$\begin{aligned} \sigma_{B,\text{upd}} &\leftarrow \text{PoK}(\text{sk}_B, c := \mathcal{H}(\text{pk}'_B || \mathcal{C}'_{\text{BLISK}})) \\ \pi_{\text{upd}} &\leftarrow \Pi.\text{Prove}(\text{pp}_\Pi, \mathcal{R}_{\text{upd}}, \text{sk}'_B) \end{aligned}$$

The relation $\mathcal{R}_{\text{upd}}$ in this case is:

$$\mathcal{R}_{\text{upd}} = \left\{ \begin{array}{c|c} v'_B, (v_B, v_C), (\text{pk}'_{v_v}, \text{aux}'_{v_v}) & \begin{array}{l} v_B = [\text{sk}_B]G \\ \text{aux}'_{v_v} = (v'_B, v_C) \end{array} \\ \text{sk}_B, \text{sk}'_B & \begin{array}{l} \text{sk}'_{v_v} = \mathcal{H}_{\text{art}}([\text{sk}'_B]v_C) \\ \text{pk}'_{v_v} = [\text{sk}'_{v_v}]G \end{array} \end{array} \right\}$$

If $\Sigma.\text{SigVerify}(\text{pk}'_B, m, \sigma_{B,\text{upd}}) = 1$ and $\Pi.\text{Verify}(\text{vp}, \mathcal{R}_{\text{upd}}, \pi_{\text{upd}}) = 1$, the circuit updates with $v_B \leftarrow \text{pk}'_B$, $v_v \leftarrow (\text{pk}'_{v_v}, \text{aux}'_{v_v})$ and $\text{pk}_r = \text{pk}_A + \text{pk}'_{v_v}$. We see that Alice's key is still required to authorize the signature and Carol can recalculate the secret as $\text{sk}_v \leftarrow \text{Derive}(\text{pp}_K, \text{sk}_C, \text{aux}'_{v_v})$.

A.2 BLISK Security

We analyze the security of $\Sigma^{\Sigma+K+\Pi}_{\text{BLISK}}$ in the EUF-CMA model. Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary. We first formalize the security game, then proceed via a sequence of hybrid games with explicit probability bounds.

A.2.1 Security Game Formalization

Definition A.1 (BLISK EUF-CMA Game). *The security game $\text{Game}_{\text{BLISK}}^{\text{EUF-CMA}}(\mathcal{A}, \lambda)$ proceeds as follows:*

1. **Setup:** Challenger $\mathcal{C}$ runs $\text{pps} \leftarrow \text{Setup}(1^\lambda)$ and generates $(\text{sk}_u, \text{pk}_u) \leftarrow \text{KeyGen}(1^\lambda)$ for all $u \in \mathcal{U}$.
2. **Corruption:** $\mathcal{A}$ specifies $S_{\text{corr}} \subseteq \mathcal{U}$ and receives $\{\text{sk}_u : u \in S_{\text{corr}}\}$.
3. **Policy:** $\mathcal{A}$ outputs circuit $\mathcal{C}$ with $\text{Eval}(\mathcal{C}, S_{\text{corr}}) = \text{False}$.
4. **Compilation:** $\mathcal{C}$ compiles $\mathcal{C}_{\text{BLISK}}$ and provides all public states $\{(\text{pk}_v, \text{aux}_v, \pi_v)\}_{v \in \mathcal{V}}$.
5. **Queries:** $\mathcal{A}$ makes signing queries (m_i, Q_i) where $\text{Eval}(\mathcal{C}, Q_i) = \text{True}$. Challenger simulates honest parties.
6. **Forgery:** $\mathcal{A}$ outputs (m^*, σ^*) and wins if $\text{SigVerify}(\text{pk}_r, m^*, \sigma^*) = 1$ and $m^* \notin \{m_i\}$.

A.2.2 Cryptographic Assumptions

For completeness, we state the assumptions referenced in the proof:

- **DDH:** For $a, b, c \xleftarrow{\$} \mathbb{F}_q$, distinguishing $([a]G, [b]G, [ab]G)$ from $([a]G, [b]G, [c]G)$ is hard.

- **PRF-ODH:** For $\mathcal{H}_{\text{art}}$ modeled as a random oracle, $H_{\text{art}}([ab]G)$ is indistinguishable from random given $([a]G, [b]G)$.
- **AOMDL:** No algebraic adversary can compute $k+1$ discrete logarithms given k DL oracle queries.
- **Π -Soundness:** For the proof system Π , no PPT prover can produce a valid proof for a false statement except with negligible probability.

A.2.3 Main Theorem

Theorem A.1. *Let $\text{BLISK}_{\mathcal{P}}$ be the BLISK protocol for policy $\mathcal{P}$ instantiated with group $\mathbb{G}$ of prime order q , hash functions $\mathcal{H}_{\text{sig}}, \mathcal{H}_{\text{art}}$ (random oracles), multisignature scheme Σ , key agreement $\mathcal{K}$, and proof system Π . Then $\Sigma_{\text{BLISK}}^{\Sigma+K+\Pi}$ is EUF-CMA secure, and for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\Sigma_{\text{BLISK}}^{\Sigma+K+\Pi}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \text{Adv}_{\Sigma}^{\text{EUF-CMA}}(\mathcal{B}) + n_{\vee} \cdot \left(\text{Adv}_{\mathbb{G}}^{\text{DDH}}(\mathcal{A}) + \text{Adv}_{\mathbb{G}, \mathcal{H}_{\text{art}}}^{\text{PRF-ODH}}(\mathcal{A}) \right) + n_{\vee} \cdot \text{Adv}_{\Pi}^{\text{sound}}(\mathcal{A}) \quad (1)$$

where $n_{\vee}$ is the number of OR gates in the policy circuit.

Proof. We proceed via a sequence of games. Let S_i denote the event that $\mathcal{A}$ outputs a valid forgery in Game i .

Game 0 (Real World). This is the real BLISK EUF-CMA game.

- *Setup:* $\mathcal{C}$ generates keys for honest users. $\mathcal{A}$ selects corrupted set $S_{\text{corr}} \subseteq \mathcal{U}$ such that $\text{Eval}(\mathcal{C}, S_{\text{corr}}) = \text{False}$. The circuit $\mathcal{C}_{\text{BLISK}}$ is constructed: for each OR gate, the secret is derived as $\text{sk}_v = \mathcal{H}_{\text{art}}([\text{sk}_L]\text{pk}_R)$ and $\text{pk}_v = [\text{sk}_v]G$.
- *Queries:* $\mathcal{A}$ makes signing queries. $\mathcal{C}$ uses real secret keys of honest nodes.
- *Forgery:* $\mathcal{A}$ outputs (m^*, σ^*) .

By definition: $\text{Adv}_{\Sigma_{\text{BLISK}}^{\Sigma+K+\Pi}}^{\text{EUF-CMA}}(\mathcal{A}) = \Pr[S_0]$.

Game 1 (Enforce Proof Verification). We modify the game to abort if any OR-gate proof fails verification.

Change: During policy acceptance, whenever an OR-node public state $(\text{pk}_v, \text{aux}_v)$ is submitted with proof π_v , the challenger verifies $\Pi.\text{Verify}(\text{vp}, R_v, \pi_v)$ and aborts if verification fails.

Analysis: Define failure event F_{proof} : “an OR-node is accepted with an invalid or missing proof.” By the knowledge soundness of Π :

$$\Pr[F_{\text{proof}}] \leq n_{\vee} \cdot \text{Adv}_{\Pi}^{\text{sound}}(\mathcal{A})$$

where we apply a union bound over all $n_{\vee}$ OR gates.

Since valid proofs attest that pk_v was correctly derived from child keys via $K.\text{Derive}$ and $K.\text{Resolve}$, this game is identical to Game 0 when all proofs are valid:

$$|\Pr[S_0] - \Pr[S_1]| \leq n_{\vee} \cdot \text{Adv}_{\Pi}^{\text{sound}}(\mathcal{A})$$

Game 2 (Randomizing Unsatisfied OR Gates). We replace derived keys at OR gates not controlled by the adversary with uniformly random keys.

Change: Define predicate $\text{Sat}(v, S_{\text{corr}})$ which is **True** iff S_{corr} can derive sk_v . Since $\text{Eval}(\mathcal{C}, S_{\text{corr}}) = \text{False}$, we have $\text{Sat}(v, S_{\text{corr}}) = \text{False}$.

For each OR gate v where $\text{Sat}(v, S_{\text{corr}}) = \text{False}$, the adversary controls at most one child (otherwise they could derive the shared secret). Let the children be v_L, v_R . The challenger:

1. Samples $\text{sk}_v \xleftarrow{\$} \mathbb{F}_q$ uniformly at random
2. Sets $\text{pk}_v = [\text{sk}_v]G$
3. Programs $\mathcal{H}_{\text{art}}$ to be consistent (lazy sampling)

Analysis: We show this is indistinguishable via a hybrid argument over OR gates. Consider a single OR gate v with children v_L, v_R where $\mathcal{A}$ controls neither child's secret key.

Lemma A.2 (OR-Gate Indistinguishability). *For an OR gate v where $\mathcal{A}$ does not control both children, replacing $\text{sk}_v = \mathcal{H}_{\text{art}}([\text{sk}_L]\text{pk}_R)$ with $\text{sk}_v \xleftarrow{\$} \mathbb{F}_q$ is indistinguishable under DDH and PRF-ODH.*

Proof of Lemma A.2. The derived secret is $\text{sk}_v = H_{\text{art}}([\text{sk}_L \cdot \text{sk}_R]G)$. Given only $\text{pk}_L = [\text{sk}_L]G$ and $\text{pk}_R = [\text{sk}_R]G$:

1. By DDH, $[\text{sk}_L \cdot \text{sk}_R]G$ is indistinguishable from $[c]G$ for random c .
2. By PRF-ODH, $\mathcal{H}_{\text{art}}([c]G)$ is indistinguishable from a random value when $\mathcal{H}_{\text{art}}$ is a random oracle and $\mathcal{A}$ cannot query $\mathcal{H}_{\text{art}}$ on $[c]G$ directly.

Thus, sk_v is indistinguishable from uniform. $\square$

Applying Lemma A.2 to each unsatisfied OR gate via a standard hybrid argument:

$$|\Pr[S_1] - \Pr[S_2]| \leq n_v \cdot \left(\text{Adv}_{\mathbb{G}}^{\text{DDH}}(\mathcal{A}) + \text{Adv}_{\mathbb{G}, \mathcal{H}_{\text{art}}}^{\text{PRF-ODH}}(\mathcal{A}) \right)$$

Signing Oracle Simulation: Crucially, since $\mathcal{C}$ knows the (now random) scalars sk_v for all unsatisfied nodes, it can still answer signing queries correctly by using these scalars in the Σ .SigGen protocol.

Game 3 (Root Key is Random). After Game 2, we observe that the root public key pk_r is now derived from randomized components.

Claim: In Game 2, pk_r is uniformly distributed in $\mathbb{G}$.

Proof of Claim: By the CNF structure, $\text{pk}_r = \Sigma.\text{KeyAgg}(\{\text{pk}_v : v \in \mathcal{V}_{\text{max}}^\vee\})$. Since at least one clause in the CNF contains an OR gate where $\mathcal{A}$ does not control a satisfying subset (otherwise $\text{Eval}(\mathcal{C}, S_{\text{corr}}) = \text{True}$), at least one pk_v in the aggregation is uniformly random. By the key aggregation structure of MuSig2:

$$\text{pk}_r = \sum_{v \in \mathcal{V}_{\text{max}}^\vee} [a_v]\text{pk}_v$$

where $a_v = \mathcal{H}_{\text{agg}}(L, \text{pk}_v)$. Since at least one pk_v is uniform and independent, pk_r is uniform over $\mathbb{G}$.

This game is identical to Game 2:

$$\Pr[S_2] = \Pr[S_3]$$

Game 4 (Reduction to Σ). We construct a simulator $\mathcal{B}$ that breaks Σ if $\mathcal{A}$ wins.

Construction of $\mathcal{B}$:

1. $\mathcal{B}$ receives challenge public key pk_Σ from the EUF-CMA challenger $\mathcal{C}_\Sigma$.
2. $\mathcal{B}$ sets $\text{pk}_r \leftarrow \text{pk}_\Sigma$. This is consistent with Game 3 where pk_r is uniformly random.
3. *Key Generation:* $\mathcal{B}$ generates keys for corrupted parties honestly. For honest parties, $\mathcal{B}$ simulates key generation but does not know the secret keys corresponding to pk_r .
4. *Signing Queries:* When $\mathcal{A}$ initiates a signing query for message m :
 - (a) $\mathcal{A}$ sends nonces/commitments as per Σ .
 - (b) $\mathcal{B}$ forwards $\mathcal{A}$'s contributions to $\mathcal{C}_\Sigma$ and receives honest nonces.
 - (c) $\mathcal{B}$ returns combined nonces to $\mathcal{A}$.
 - (d) Upon receiving $\mathcal{A}$'s partial signature, $\mathcal{B}$ forwards to $\mathcal{C}_\Sigma$.
 - (e) $\mathcal{C}_\Sigma$ returns valid signature σ . $\mathcal{B}$ returns σ to $\mathcal{A}$.
5. *Forgery:* If $\mathcal{A}$ outputs valid forgery (m^*, σ^*) on unqueried m^* , then $\mathcal{B}$ outputs (m^*, σ^*) to $\mathcal{C}_\Sigma$.

Analysis: The simulation is perfect because:

- $\mathcal{C}_\Sigma$ runs the honest signing algorithm of Σ .

- The root key distribution matches Game 3 (both uniform).
- Nonces and signature distributions are identical.

If $\mathcal{A}$ wins in Game 3, then (m^*, σ^*) satisfies:

- $\text{SigVerify}(\text{pk}_r, m^*, \sigma^*) = 1$, and since $\text{pk}_r = \text{pk}_\Sigma$, this means $\Sigma.\text{Verify}(\text{pk}_\Sigma, m^*, \sigma^*) = 1$.
- m^* was never queried by $\mathcal{A}$, so $\mathcal{B}$ never queried $\mathcal{C}_\Sigma$ on m^* .

Therefore:

$$\Pr[S_3] \leq \text{Adv}_{\Sigma}^{\text{EUF-CMA}}(\mathcal{B})$$

Combining the Games. Summing the transition bounds:

$$\begin{aligned} \text{Adv}_{\Sigma^{\Sigma+K+\Pi}}^{\text{EUF-CMA}}(\mathcal{A}) &= \Pr[S_0] \\ &\leq \Pr[S_1] + n_v \cdot \text{Adv}_{\Pi}^{\text{sound}}(\mathcal{A}) \\ &\leq \Pr[S_2] + n_v \cdot \text{Adv}_{\Pi}^{\text{sound}}(\mathcal{A}) + n_v \cdot \left(\text{Adv}_{\mathbb{G}}^{\text{DDH}}(\mathcal{A}) + \text{Adv}_{\mathbb{G}, \mathcal{H}_{\text{art}}}^{\text{PRF-ODH}}(\mathcal{A}) \right) \\ &= \Pr[S_3] + n_v \cdot \text{Adv}_{\Pi}^{\text{sound}}(\mathcal{A}) + n_v \cdot \left(\text{Adv}_{\mathbb{G}}^{\text{DDH}}(\mathcal{A}) + \text{Adv}_{\mathbb{G}, \mathcal{H}_{\text{art}}}^{\text{PRF-ODH}}(\mathcal{A}) \right) \\ &\leq \text{Adv}_{\Sigma}^{\text{EUF-CMA}}(\mathcal{B}) + n_v \cdot \text{Adv}_{\mathbb{G}}^{\text{DDH}}(\mathcal{A}) + n_v \cdot \text{Adv}_{\mathbb{G}, \mathcal{H}_{\text{art}}}^{\text{PRF-ODH}}(\mathcal{A}) + n_v \cdot \text{Adv}_{\Pi}^{\text{sound}}(\mathcal{A}) \end{aligned}$$

Assuming Σ is EUF-CMA secure, DDH and PRF-ODH hold in $\mathbb{G}$, and Π is knowledge sound, all terms on the right-hand side are negligible in λ . Therefore, $\text{Adv}_{\Sigma^{\Sigma+K+\Pi}}^{\text{EUF-CMA}}(\mathcal{A})$ is negligible. $\square$

A.2.4 Concrete Security for MuSig2 + 0-ART + Bulletproofs

For the specific instantiation $\Sigma_{\text{BLISK}}^{\text{MuSig2}[v=4], 0\text{-ART}[\text{BP}]}$ described in Section ??:

Corollary A.2.1. *The instantiation with MuSig2, 0-ART, and Bulletproofs over secp256k1 satisfies:*

$$\text{Adv}_{\text{BLISK}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \text{Adv}_{\text{secp256k1}}^{\text{AOMDL}}(\mathcal{B}) + n_v \cdot \text{Adv}_{\text{secp256k1}}^{\text{DDH}}(\mathcal{A}) + n_v \cdot \text{Adv}_{\text{secp256k1}, \mathcal{H}_{\text{art}}}^{\text{PRF-ODH}}(\mathcal{A}) + n_v \cdot \text{Adv}_{\text{secp256k1}}^{\text{DL}}(\mathcal{A})$$

where the last term bounds Bulletproofs soundness under the discrete logarithm assumption.

A.2.5 Discussion of Security Guarantees

Adaptive vs. Static Corruption. The proof above considers static corruption where $\mathcal{A}$ commits to S_{corr} before seeing public keys. Extending to adaptive corruption requires additional techniques (e.g., complexity leveraging or strong assumptions) and is left for future work.

Key Update Security. The KeyUpdate operation preserves security provided:

1. The update proof π_{upd} verifies under Π .
2. The old key holder authorizes the update (via $\sigma_{B, \text{upd}}$).

An adversary cannot inject malicious keys without possessing the corresponding secret, as this would require forging either the authorization signature or the proof.

Coordinator Trust. The coordinator in BLISK is assumed to be honest-but-curious: they facilitate communication but cannot forge signatures or manipulate the policy circuit beyond what public verification catches. Malicious coordinators can only cause denial-of-service by refusing to publish valid states, which is detectable.